Operator Overloading

1. Binary Operator Overloading

- Used for operators like `+`, `-`, `*`, `/`, etc., between two operands.
- Implemented as a member or friend function.
- Syntax for member function:
- `ClassName operator+(const ClassName& obj);`
- Example:
- `Result = Obj1 + Obj2;`

2. Unary Operator Overloading

- Used for operators like `++`, `--`, `-`, etc., with one operand.
- Implemented as a member function.
- Syntax:
- `ClassName operator++(); // Pre-increment`
- `ClassName operator++(int); // Post-increment`
- Example:
- `++Obj;  // Pre-increment`
- `Obj++;  // Post-increment`

3. Relational and Logical Operator Overloading

- Used for operators like `==`, `!=`, `<`, `>`, `&&`, `||`, etc.
- Typically returns `bool`.
- Syntax:
- `bool operator==(const ClassName& obj);`
- Example:
- `if (Obj1 == Obj2) { ... }`

4. Operator Overloading Using Friend Functions

- Used for operations that need access to private/protected members but are not member functions.
- Syntax:
- `friend ReturnType operator+(const ClassName& obj1, const ClassName& obj2);`
- Example:
- `friend Complex operator+(const Complex& c1, const Complex& c2);`

5. Limitations of Operator Overloading

- Cannot create new operators.
- Cannot overload `::`, `.*`, `.`, `sizeof`, `typeid`.
- Overloading might confuse readability if used excessively.
- Default precedence and associativity cannot be changed.

Inheritance

1. Defining Derived Class

- Syntax:
- `class DerivedClass : accessSpecifier BaseClass { ... };`
- Access specifier (`public`, `protected`, `private`) affects the accessibility of base class members in the derived class.

---

2. Single Inheritance

- One base class, one derived class.
- Example:
- `class Car : public Vehicle { ... };`

---

3. Multiple Inheritance

- Derived class inherits from multiple base classes.
- Syntax:
- `class Derived : public Base1, public Base2 { ... };`
- Risk: Ambiguity if base classes have members with the same name.

---

4. Multilevel Inheritance

- A class is derived from another derived class.
- Example:
- `class Vehicle { ... };`
- `class Car : public Vehicle { ... };`
- `class SportsCar : public Car { ... };`

---

5. Hierarchical Inheritance

- Multiple derived classes inherit from a single base class.
- Example:
- `class Vehicle { ... };`
- `class Car : public Vehicle { ... };`
- `class Truck : public Vehicle { ... };`

---

6. Virtual Base Classes

- Resolves ambiguity in **diamond problem** in multiple inheritance.
- Syntax:
- `class A { ... };`
- `class B : virtual public A { ... };`
- `class C : virtual public A { ... };`
- `class D : public B, public C { ... };`

---

7. Constructors in Derived Class

- Base class constructor is called before the derived class constructor.
- Use initialization list to call specific constructors:
- `Derived(int a, int b) : Base(a) { ... };`
- Default constructors are invoked automatically if not explicitly specified.

---

8. Nesting of Classes

- Defining one class inside another.
- Inner class can access the outer class's private/protected members.
- Example:
- `class Outer {`
- `    class Inner { ... }; // Nested class`
- `};`

Virtual Functions

1. Pointers to Derived Classes

- A base class pointer can point to derived class objects.
- Example:
- `Base* ptr = new Derived();`
- `ptr->function();  // Calls the base or derived class function based on whether it's virtual.`

---

2. Applying Polymorphism using Virtual Functions

- Achieved by using **virtual** keyword in the base class.
- Allows derived class methods to override base class methods.
- Example:
- ```
  class Base {
      virtual void display() { cout << "Base class"; }
  };
  ```

---

3. Polymorphic Class

- A class with at least one virtual function is called a **polymorphic class**.
- Enables runtime polymorphism.

---

4. Pure Virtual Functions

- A virtual function with no definition, forcing derived classes to implement it.
- Syntax:
- ```
  virtual void display() = 0;  // Pure virtual function
  ```

---

5. Abstract Classes

- A class containing at least one pure virtual function.
- Cannot be instantiated.
- Example:
- ```
  class Abstract {
      virtual void func() = 0;  // Pure virtual function
  };
  ```

---

6. Early Binding (Compile-Time Binding)

- Function call is resolved at compile time.
- Used for non-virtual functions or static methods.
- Example:
- ```
  obj.function();  // Direct call, no virtual mechanism
  ```

---

7. Late Binding (Run-Time Binding)

- Function call is resolved at runtime.
- Enabled by virtual functions and base class pointers/references.
- Example:
- ```
  Base* ptr = new Derived();
  ptr->function();  // Virtual mechanism determines the function to call
  ```
  ```

Template, Exception Handling & STL

1. Generic Functions (Templates)

- **Function Templates**: Allows creating generic functions that work with any data type.
- Syntax:

```
template <typename T>
T add(T a, T b) {
    return a + b;
}
```

- Example usage:

```
int result = add(5, 10);  // T is deduced as int
double result2 = add(5.5, 4.5);  // T is deduced as double
```

---

2. Exception Handling

- **Try-Catch Blocks**: Used to catch and handle exceptions to prevent program crashes.

```
try {
    // Code that may throw exceptions
    if (x == 0) throw "Division by zero!";
}
catch (const char* msg) {
    cout << "Error: " << msg << endl;
}
```

- **throw**: Used to throw an exception when an error occurs.
  Example:

```
throw 404;  // Throwing an integer exception
```

- **Standard Exception Types**:
  - `std::exception`
  - `std::out_of_range`
  - `std::invalid_argument`

---

3. Vector (from STL)

- **Vector**: A dynamic array that grows as elements are added.
- Syntax to declare:

```
#include <vector>
std::vector<int> vec;
```

- **Common operations**:
  - `push_back(value)`: Adds an element to the end.
  - `size()`: Returns the number of elements.
  - `at(index)`: Accesses an element with bounds checking.
  - `push_back()` vs `emplace_back()`:
    - `emplace_back()` constructs the object in-place, avoiding a copy.

- Example usage:
- `std::vector<int> vec;`
- `vec.push_back(10);`
- `vec.push_back(20);`
- `cout << vec[0] << endl;  // Outputs 10`
- `cout << vec.size() << endl;  // Outputs 2`

C++ I/O System

1. Formatted I/O

- **`cin`** (input stream) and **`cout`** (output stream) are used for basic I/O operations.
- Formatted I/O controls the appearance of input and output using **setprecision**, **setw**, etc.
- **Example:**
- `#include <iostream>`
- `using namespace std;`
-
- `int main() {`
- `    float pi = 3.14159;`
- `    cout << "Pi is: " << fixed << setprecision(2) << pi << endl;  // Output: 3.14`
- `    return 0;`
- `}`
    - **`fixed`**: Forces fixed-point notation (not scientific).
    - **`setprecision(n)`**: Specifies number of digits after the decimal point.

---

2. Manipulators

- Used to modify the output format of data.
- **Common manipulators**:
    - `setw(n)`: Sets the width of the output.
    - `setfill(c)`: Sets the fill character for unused space in `setw`.
    - `left`, `right`: Aligns text (left or right).
    - `fixed`: Fixes the decimal point.
    - `scientific`: Forces scientific notation.
- **Example**:
- `#include <iostream>`
- `#include <iomanip>`
- `using namespace std;`
-

```
int main() {
    cout << setw(10) << 123 << endl;  // Output: "       123"
    cout << setfill('*') << setw(10) << 123 << endl;  // Output: "******123"
    cout << left << setw(10) << "Left" << endl;  // Output: "Left      "
    return 0;
}
```

---

3. File Read-Write-Display

- C++ supports file operations using **fstream**.
  - **`ifstream`**: Used to read from files.
  - **`ofstream`**: Used to write to files.
  - **`fstream`**: Used for both reading and writing to a file.
- **Opening Files**:
- `ifstream inFile("input.txt");  // For reading`
- `ofstream outFile("output.txt");  // For writing`
- **Reading and Writing**:
  - **Write**:
  - `outFile << "Hello, File!" << endl;`
  - **Read**:
  - `string data;`
  - `inFile >> data;`
  - `cout << data;`
- **Example (Read-Write)**:

```
#include <iostream>
#include <fstream>
using namespace std;

int main() {
    // Write to file
    ofstream outFile("example.txt");
    if (outFile.is_open()) {
        outFile << "This is a test file." << endl;
        outFile.close();
    }

    // Read from file
    ifstream inFile("example.txt");
    string line;
    if (inFile.is_open()) {
        while (getline(inFile, line)) {
            cout << line << endl;
        }
        inFile.close();
    }
    return 0;
```

- `}`
- **Closing Files**:
  Always close the file after completing operations:
- `inFile.close();`
- `outFile.close();`

Key Points Recap:

- **Formatted I/O** allows control over output precision and formatting.
- **Manipulators** modify the appearance of data in the output stream.
- **File I/O** involves reading and writing files with `fstream` (`ifstream` for reading, `ofstream` for writing).